GDD Draft — Soumois Pierre-Emmanuel — Echoes of the Vault

Assignment #1 — ELVTR Multi-Agent AI for Game Development

Executive Summary

Working Title: Echoes of the Vault

Pitch: A top-down survivor-like auto-shooter where the arena itself is a weapon — aiming means taking down an enemy just as often as blowing up a wall or a barrel to redirect the fight.

Genre / inspiration: Roguelite auto-shooter (inspired by *Deep Rock Galactic: Survivor*), with one key difference: manual aim instead of full auto-targeting.

Platform / audience: PC (Windows), single-player. Target audience: auto-shooter and roguelite-arcade players, casual to mid-core.

Design pillars:

Pillar	Description
Gamefeel / juice	Every shot, impact, and destruction must feel satisfying in the moment.
Manual aim + auto-fire	The weapon fires on its own, the player chooses the target; extended to deployables (turret): the player aims and chooses when to place it, unlike the automatic, uncontrollable placement in the reference game.
“The arena is a weapon”	Destructible terrain isn't cosmetic: it blocks sightlines, punishes enemy clustering, and generates as an active tactical layer, not mere set dressing.

Project framing: The minimal playable scope was validated with a “48h, 2 devs” test (see Game Mechanics).

Game Mechanics

Playable core (validated by the “48h, 2 devs” test):

- Base movement + manual aim + auto-fire
- 3 classic weapons, including a ground-placeable turret (aim + timing chosen by the player)
- 3 enemy types
- Explosive melee enemy: charges at the player, detonates on proximity if not avoided in time, or on death if killed at range — same radius and damage either way, hitting terrain/enemies/player without distinction.
- Simple destructible terrain: HP-based, no intermediate state, disappears at 0 HP
- 1 explosive barrel: area damage + particle effect on destruction

- In-run temporary upgrades only (persistent meta-progression between runs is explicitly excluded from this core, added later)

Level structure:

- A progress bar advances over time and speeds up with every enemy kill.
- Events punctuate the level: monster waves, chest spawns.
- Default end objective: **hold a zone/switch for X seconds** — chosen over the alternatives (boss, endless wave) because it forces a stationary defense where destructible terrain becomes a critical survival tool, directly reinforcing the “arena is a weapon” pillar.
- Objective variants (post-A#1): boss (base enemy with more HP), survive X time against an endless wave.

Level end — risk/reward window: once the objective is met, an exit spawns far from the player; they have X seconds to reach it. In the meantime, they can keep collecting XP and resources at the risk of not making it in time — no abrupt ending the moment the objective is completed.

Progression economy: temporary upgrades chosen during the run (stack for the run's duration); persistent meta-progression between runs planned for a later iteration, outside the initial core.

AI Architecture

Core constraint: AI is used only as a development-time tool (dev-time) — no AI calls at runtime. Level generation is a deterministic C++ algorithm (via Unreal's PCG Framework), designed with AI assistance, but runs entirely on its own in-game with no model calls.

Orchestration model: a single Orchestrator receives objectives (given by me, acting as Lead Architect), splits the task, and delegates to the relevant specialized agents — no generic crew.

Specialized agents:

Agent	Role
Orchestrator	Splits tasks, chooses which agent to invoke.
C++ agent	Unreal C++ code and conventions; does its own research in the existing codebase (project small enough not to pollute its context).
BP agent	Conventions and manipulation of Unreal assets/Blueprints (UI/UMG included).
Level-Design agent	During prototyping: designs fixed level layouts (concrete positions); once the generator is in place: defines generation rules (destructible density, spacing constraints) that the C++ agent implements.
Balance agent (ephemeral)	Invoked only once playtest logs are available, adjusts numeric values (HP, damage, drop rates) from real data.
Text/content agent (lightweight, occasional)	Generates upgrade text, enemy codex entries, and UI copy, with a style-consistency pass.

Project vault: a single vault dedicated to the project (tasks, GDD, technical documentation).

Technical Strategy

Engine: Unreal Engine 5 — chosen for two native systems the game directly relies on: **Chaos Destruction** (destructible terrain) and the **PCG Framework** (level generation).

Code split: gameplay logic mostly in C++; Blueprints stay a thin layer — classes inheriting from C++ for scene placement, some events calling C++ functions directly, a few visual effects (camera shake) handled in BP.

Token budget: no upfront figure; estimated task by task and corrected after real-world feedback (agile retrospective logic), per the method recommended in the course.

Logging system (for the balance agent): a file structured per run, sub-structured per level, containing: number of destroyed terrain pieces, damage dealt per weapon, kills by enemy type and by source (weapon vs terrain), upgrades chosen (order and frequency), time spent per level, damage taken and its source, XP collected, level outcome (death / exit reached in time / failure), time remaining on reaching the exit, resources collected during the risk window, encountered events and their outcome.